

Objectives:

1. To perform load flow study of the given 13 bus power system.
2. To create the necessary Excel data file for bus data, generator data, branch data and transformer data.
3. To read the system data from the excel file and use it in MATLAB rather than using hard coded data.
4. To construct the Y-bus admittance of the system using the branch and transformer data.
5. To carry out Load flow analysis by Gauss Seidel method.
6. To carry out load flow by Newton-Raphson method.
7. To compute the critical load flow quantities such as bus voltages, bus voltage angles, real/ reactive power.
8. To obtain line flows and line losses of the 13-bus power system.
9. To compare the result of the Gauss-Seidel and Newton-Raphson method.
10. To decide which load flow technique provides the better solution from the angle of convergence, accuracy, mismatch error and total losses.
11. To connect a solar PV source of 50 MW to different non-slack buses and observe the effect on system losses.
12. To determine the best bus for solar PV connection where the cumulative line loss becomes minimum.
13. To construct and simulate the same 13-bus system in PV simulator / PowerWorld.
14. To compare MATLAB load flow results with simulator results.
15. To observe the effect of solar PV integration on voltage profile and total system loss.
16. To identify the difficulties faced during load flow analysis and simulation.
17. To suggest possible solutions for the problems encountered during the project.
18. To develop a complete understanding of practical power system load flow analysis and renewable energy integration.

Theory:

1. Load Flow Analysis

Load flow study is one of the most important studies in power system operation and planning. It is used for finding a steady-state operating point of an electrical power network. A power system consists of various buses which are interconnected by transmission lines, transformers, generators, and loads. The primary objective of load flow analysis is to determine the voltage magnitude, voltage angle, active power, reactive power, power flow along the lines and line losses of the system. This project is about applying load flow analysis on a 13 bus power system using PowerWorld. The system consists of various buses, transmission lines, transformer branches, loads and generators. Two numerical techniques, Gauss-Seidel and Newton-Raphson method are used for solving load flow problems. Load flow solution is obtained with both the methods with load flow results compared to find out which method gives better load flow solution. In this project, a Solar PV placement is also used to minimize total system line loss.

2. Per-Unit System

The Per-unit system is used widely in power system analysis to simplify the computation and to make the values more easily comparable. In this system, actual electrical quantities (voltage, current, impedance, power etc) are presented in fractional form of some chosen base quantities. The general formula of PU system:

$$\text{Per Unit Value} = \frac{\text{Actual value}}{\text{Base value}}$$

For this project, the system base is 100 MVA. Since the 13-bus system contains different voltage levels such as 138 kV and 66 kV, the per-unit system helps keep all calculations consistent. It is especially useful for forming the Y-bus matrix and solving the load flow equations.

3. Bus Classification

During the load flow analysis, buses are primarily divided into three categories: slack bus, PV bus and PQ bus. The amount of each type of bus is known and unknown. The classification of the buses is significant because the load flow equations are solved differently for different types of buses. The slack bus is the reference bus, the PV bus is typically a generator bus and the PQ bus is typically a load bus

Bus Type	Known Quantities	Unknown Quantities	Function
Slack bus	Voltage magnitude and angle	Real and reactive power	Balances system and power loss
PV bus	Real power and voltage magnitude	Reactive power and angle	Generator bus
PQ bus	Real and reactive power	Voltage magnitude and angle	Load bus

4. Y-Bus Matrix

The Y-bus matrix, or bus admittance matrix, represents the electrical connection between different buses of a power system. It is formed using the resistance, reactance, and line charging data of transmission lines and transformers. The Y-bus matrix is required for both the Gauss-Seidel and Newton-Raphson load flow methods. For a transmission line having impedance,

$$Z=R+jX$$

the corresponding admittance is:

$$Y = \frac{1}{Z}$$

The diagonal elements of the Y-bus matrix represent the total admittance connected to a bus. The off-diagonal elements represent the negative of the admittance between two connected buses. If the Y-bus matrix is incorrect, the final load flow result will also be incorrect.

5. Power Flow Equations

The complex power injected at bus i is expressed as $S_i = P_i + jQ_i$, where P_i is real power and Q_i is reactive power. The complex power can also be written as $S_i = V_i I_i^*$, where V_i is the bus voltage and I_i^* is the conjugate of the bus current.

6. Gauss-Seidel Method

Gauss-Seidel method is an iterative method for solving Load Flow problems. A typical assumption is a flat start, in which case unknown bus voltage magnitudes are assumed to be 1.0 pu and voltage angles assumed to be 0 degrees. This approach involves successive updating of the voltage of each bus from the most recent available voltage values. The process is repeated until the error/power mismatch is less than the desired tolerance. The method is simple and easy to implement on MATLAB, but it generally takes more iterations and converging speed is not fast for large and heavily loaded systems. The Gauss-Seidel method can be used to appreciate the fundamental principle of the iterative load flow analysis. It might be sensitive to the initial guess of voltage, however, and might have convergence issues when the system condition is poor or heavily loaded.

7. Newton-Raphson Method

Newton-Raphson method is an important and popular approach for solving load flow problems. At each iteration, it updates the voltage magnitudes and the voltage angles by using power mismatch equations and the Jacobian matrix. The real and reactive power mismatches are determined as the difference between the specified and calculated powers. Typically, the Newton-Raphson method is much faster than the Gauss-Seidel method. Due to its strong convergence behavior it is more suitable to be used in medium and large power systems. It is mathematically more complicated, however, as the Jacobian matrix has to be assembled and modified as the solution is calculated. For any practical study of a power system, Newton-Raphson is preferred since it provides accurate results for a lesser number of iterations. The comparison of the Newton-Raphson result with the Gauss-Seidel result for this project is based on the number of iterations, maximum mismatch, voltage profile, total loss, and convergence time.

9. Line Flow and Line Loss

Line flow refers to the real and reactive power flowing through transmission lines from one bus to another. During transmission, some power is lost due to the resistance and reactance of the lines. These losses reduce system efficiency and increase the required generation.

The real power loss of a line can be found from the sum of sending-end and receiving-end real power flows. Similarly, reactive power loss can be found from the corresponding reactive power flows. Total system loss is obtained by adding the losses of all transmission lines and transformer branches. Reducing line loss is important because it improves efficiency, reduces operating cost, and improves the overall performance of the power system.

10. Solar PV Integration

Solar PV is a renewable energy source, which generates electrical energy from sunlight. Solar PV can be considered as a distributed generation source in a power system. Line losses, voltage profile and loading on transmission lines can be optimized by placing solar PV. In this project, an available solar PV source of 50MW is available. The PV solar source needs to be connected to an appropriate non-slack bus and the overall line loss of the system needs to be minimized. In order to determine the most suitable bus, the PV source is sequentially placed on each bus, load flow for each case is done and the total line losses are obtained. The bus with the smallest total line loss is used as the best location to integrate PV.

11. Load Power Factor

Power Factor is the angle between the voltage and current. It is a value of the cosine of the angle between real power and apparent power. When the ratio is less than one, load consumes the reactive power, this is called lagging power factor and it is commonly found in inductive loads.

In this project, the load power factor at Bus 7 must be maintained at 0.9 lagging. If the real power at Bus 7 is known, the reactive power can be calculated using $Q = P \tan(\cos^{-1}(0.9))$.

12. PowerWorld Simulation

The results obtained from the MATLAB load flow program can be verified using PowerWorld. The 13-bus power system is created in the Simulator based on the following data: Bus data, Generator data, Branch data, Transformer data. Once the load flow simulation is carried out, the voltage magnitude, voltage angle, real power, reactive power and line losses are recorded. Then, the results of the simulator are compared with the results of the MATLAB. Due to rounding, tolerance and solver settings and assumptions of the base values, the differences are small. If the voltage profile, direction of the power flow and total losses are close, however, the MATLAB result can be considered valid.

13. Importance of Loss Minimization

Minimization of losses is an important aspect of the operation of a power system. High line loss is the result of increased generation of power to meet the same load demand. This will lead to a higher running cost and lower system efficiency. Solar PV can be located at a suitable bus where it can serve local demand and lessen the need to flow power over long transmission lines. Hence, placing solar PV is not only a problem of renewable energy integration, but also an optimization problem. The best PV location should be selected based on minimum total line loss and improvement in the voltage profile.

Softwares used:

MATLAB & PowerWorld simulator

Answer to the question “A”

Matlab code for load flow studies by Gauss-Seidel and Newton-Raphson methods;

```
% Student ID: 21321070 | K=7 | A0=50MW
clc;
clear;
close all;
Sbase = 100; % MVA
Vbase_HV = 138; % kV (buses 1-5)
Vbase_LV = 66; % kV (buses 6-13)
%% ---- BUS DATA -----
% Col: [Bus | Type | PL(MW) | QL(MVAR) | Pg(MW) | Vsp(pu) | Qmax |
Qmin]
% Type: 1=Slack, 2=PV, 3=PQ
busData = [
    1 1 0 0 0 1.06 Inf -Inf;
    2 2 0 0 100 1.02 50 -40;
    3 3 95 20 0 1.00 0 0;
    4 3 45 0 0 1.00 0 0;
    5 3 10 2 0 1.00 0 0;
    6 3 11 7.5 0 1.00 0 0;
    7 3 7 3.39 0 1.00 0 0;
    8 3 25 7 0 1.00 0 0;
    9 3 20 6 0 1.00 0 0;
    10 3 20 5 0 1.00 0 0;
    11 3 30 10 0 1.00 0 0;
    12 2 13.5 6 20 1.01 100 -60;
    13 3 20 5 0 1.00 0 0;
];
nBus = size(busData,1);
Psch = (busData(:,5) - busData(:,3)) / Sbase;
Qsch = -busData(:,4) / Sbase;
lineData = [
    1 2 0.01938 0.05917 0.0528;
    1 5 0.05403 0.22304 0.0492;
    2 3 0.04699 0.19797 0.0438;
    2 4 0.05811 0.17632 0.0340;
    2 5 0.05695 0.17388 0.0346;
    3 4 0.06701 0.17103 0.0128;
    4 5 0.01335 0.04211 0.0000;
    6 10 0.09498 0.19890 0.0000;
    6 11 0.12291 0.25581 0.0000;
    6 12 0.06615 0.13027 0.0000;
    7 8 0.00000 0.11010 0.0000;
    8 9 0.03181 0.08450 0.0000;
    8 13 0.12711 0.27038 0.0000;
    9 10 0.08205 0.19207 0.0000;
    11 12 0.22092 0.19988 0.0000;
```

```

12 13    0.17093  0.34802  0.0000;
];
xfmrData = [
    4  7    0    0.20912  0;
    4  8    0    0.55618  0;
    5  6    0    0.25202  0;
];
branchData = [lineData; xfmrData];
nBranch = size(branchData,1);

Ybus = zeros(nBus, nBus);
for k = 1:nBranch
    p = branchData(k,1);
    q = branchData(k,2);
    r = branchData(k,3);
    x = branchData(k,4);
    b = branchData(k,5);
    z = r + 1j*x;
    y = 1/z;
    Ybus(p,q) = Ybus(p,q) - y;
    Ybus(q,p) = Ybus(q,p) - y;
    Ybus(p,p) = Ybus(p,p) + y + 1j*b/2;
    Ybus(q,q) = Ybus(q,q) + y + 1j*b/2;
end
G = real(Ybus);
B = imag(Ybus);
fprintf('\n===== GAUSS-SEIDEL METHOD =====\n');
maxIter_GS = 1000;
tol_GS      = 1e-5;
accel       = 1.6;
V_GS = busData(:,6) .* exp(1j*0);
slackBus = find(busData(:,2)==1);
pvBus    = find(busData(:,2)==2);
pqBus    = find(busData(:,2)==3);
iter_GS = 0;
converged_GS = false;
for iter = 1:maxIter_GS
    V_prev = V_GS;

    for i = 1:nBus
        if busData(i,2) == 1, continue; end

        sum_YV = 0;
        for j = 1:nBus
            if j ~= i
                sum_YV = sum_YV + Ybus(i,j)*V_GS(j);
            end
        end
    end
end

```

```

P_i = Psch(i);
Q_i = Qsch(i);

if busData(i,2) == 2
    Q_calc = -imag(conj(V_GS(i)) * sum(Ybus(i,:).*V_GS.));
    Q_calc = Q_calc + imag(Ybus(i,i))*abs(V_GS(i))^2
    Q_calc = imag(conj(V_GS(i)) * (Ybus(i,:)*V_GS));
    Qmax = busData(i,7);
    Qmin = busData(i,8);
    Q_i = max(Qmin/Sbase, min(Qmax/Sbase, -Q_calc));
end

% Gauss-Seidel update
V_new = (1/Ybus(i,i)) * ( conj((P_i - 1j*Q_i)/conj(V_GS(i))) -
sum_YV );

if busData(i,2) == 2
    V_new = busData(i,6) * V_new / abs(V_new);
end

V_GS(i) = V_prev(i) + accel*(V_new - V_prev(i));

if busData(i,2) == 2
    V_GS(i) = busData(i,6) * V_GS(i)/abs(V_GS(i));
end
end

iter_GS = iter;
if max(abs(V_GS - V_prev)) < tol_GS
    converged_GS = true;
    break;
end
end
if converged_GS
    fprintf('Converged in %d iterations.\n', iter_GS);
else
    fprintf('Did NOT converge after %d iterations.\n', maxIter_GS);
end

fprintf('\n===== NEWTON-RAPHSON METHOD =====\n');
maxIter_NR = 50;
tol_NR = 1e-6;
Vmag = busData(:,6);
delta = zeros(nBus,1);
iter_NR = 0;
converged_NR = false;

```

```

busList_delta = [pvBus; pqBus];
busList_V      = pqBus;
for iter = 1:maxIter_NR
    V_NR = Vmag .* exp(1j*delta);
    S_calc = V_NR .* conj(Ybus * V_NR);
    P_calc = real(S_calc);
    Q_calc = imag(S_calc);

    dP = Psch(busList_delta) - P_calc(busList_delta);
    dQ = Qsch(busList_V)      - Q_calc(busList_V);
    mismatch = [dP; dQ];

    if max(abs(mismatch)) < tol_NR
        converged_NR = true;
        iter_NR = iter - 1;
        break;
    end
    iter_NR = iter;

    n_delta = length(busList_delta);
    n_V      = length(busList_V);
    n_total = n_delta + n_V;
    J = zeros(n_total, n_total);

    for ii = 1:n_delta
        i = busList_delta(ii);
        for jj = 1:n_delta
            j = busList_delta(jj);
            if i == j
                J(ii,jj) = -Q_calc(i) - B(i,i)*Vmag(i)^2;
            else
                J(ii,jj) =
Vmag(i)*Vmag(j)*(G(i,j)*sin(delta(i)-delta(j)) -
B(i,j)*cos(delta(i)-delta(j)));
            end
        end
    end

    for ii = 1:n_delta
        i = busList_delta(ii);
        for jj = 1:n_V
            j = busList_V(jj);
            if i == j

```



```

        J(ii, n_delta+jj) = P_calc(i)/Vmag(i) + G(i,i)*Vmag(i);
    else
        J(ii, n_delta+jj) =
Vmag(i)*(G(i,j)*cos(delta(i)-delta(j)) +
B(i,j)*sin(delta(i)-delta(j)));
    end
end
end

for ii = 1:n_V
    i = busList_V(ii);
    for jj = 1:n_delta
        j = busList_delta(jj);
        if i == j
            J(n_delta+ii, jj) = P_calc(i) - G(i,i)*Vmag(i)^2;
        else
            J(n_delta+ii, jj) =
-Vmag(i)*Vmag(j)*(G(i,j)*cos(delta(i)-delta(j)) +
B(i,j)*sin(delta(i)-delta(j)));
        end
    end
end

for ii = 1:n_V
    i = busList_V(ii);
    for jj = 1:n_V
        j = busList_V(jj);
        if i == j
            J(n_delta+ii, n_delta+jj) = Q_calc(i)/Vmag(i) -
B(i,i)*Vmag(i);
        else
            J(n_delta+ii, n_delta+jj) =
Vmag(i)*(G(i,j)*sin(delta(i)-delta(j)) -
B(i,j)*cos(delta(i)-delta(j)));
        end
    end
end
dx = J \ mismatch;
delta(busList_delta) = delta(busList_delta) + dx(1:n_delta);
Vmag(busList_V) = Vmag(busList_V) .* (1 + dx(n_delta+1:end));
end
if converged_NR
    fprintf('Converged in %d iterations.\n', iter_NR);
else
    fprintf('Did NOT converge after %d iterations.\n', maxIter_NR);
end

```

```

V_NR = Vmag .* exp(1j*delta);

[Pij_GS,Qij_GS,Pji_GS,Qji_GS,Ploss_GS,Qloss_GS] =
calcBranchFlows(V_GS, branchData, Sbase);
[Pij_NR,Qij_NR,Pji_NR,Qji_NR,Ploss_NR,Qloss_NR] =
calcBranchFlows(V_NR, branchData, Sbase);

S_GS = V_GS .* conj(Ybus * V_GS) * Sbase;
S_NR = V_NR .* conj(Ybus * V_NR) * Sbase;

fprintf('\n=====
=\n');
fprintf('=====\\
n');
fprintf('%-4s %-5s | %-7s %-7s %-7s %-8s | %-7s %-7s %-7s %-8s\n',...
    'Bus','Type','P (MW)','Q (MVar)','|V| (pu)','δ (deg)',...
    'P (MW)','Q (MVar)','|V| (pu)','δ (deg)');
fprintf('%-4s %-5s | %-7s %-7s %-7s %-8s | %-7s %-7s %-7s %-8s\n',...
    'Bus','Type','P (MW)','Q (MVar)','|V| (pu)','δ (deg)',...
    'P (MW)','Q (MVar)','|V| (pu)','δ (deg)');
fprintf('%s\n', repmat('-',1,80));
types = {'Slack','PV','PQ'};
for i = 1:nBus
    t = types{busData(i,2)};
    fprintf('%-4d %-5s | %7.3f %7.3f %7.5f %8.4f | %7.3f %7.3f %7.5f
%8.4f\n',...
        i, t,...
        real(S_GS(i)), imag(S_GS(i)), abs(V_GS(i)),
rad2deg(angle(V_GS(i))),...
        real(S_NR(i)), imag(S_NR(i)), abs(V_NR(i)),
rad2deg(angle(V_NR(i))));
end
fprintf('\n=====
=\n');
fprintf('          LINE FLOW & LOSS RESULTS (Newton-Raphson)\n');
fprintf('=====\\
n');
fprintf('%-4s %-4s | %-9s %-9s | %-9s %-9s | %-8s %-8s\n',...

'From','To','Pij (MW)','Qij (MVar)','Pji (MW)','Qji (MVar)','Ploss (MW)','Q
loss (MVar)');
fprintf('%s\n', repmat('-',1,78));
for k = 1:nBranch
    fprintf('%-4d %-4d | %9.4f %9.4f | %9.4f %9.4f | %8.4f %8.4f\n',...
        branchData(k,1), branchData(k,2),...
        Pij_NR(k), Qij_NR(k), Pji_NR(k), Qji_NR(k),...
        Ploss_NR(k), Qloss_NR(k));
end

```

```

fprintf('\nTotal Real Power Loss (NR):      %.4f MW\n',
sum(Ploss_NR));
fprintf('Total Reactive Power Loss (NR):  %.4f MVar\n',
sum(Qloss_NR));
fprintf('Total Real Power Loss (GS):      %.4f MW\n',  sum(Ploss_GS));
fprintf('Total Reactive Power Loss (GS):  %.4f MVar\n',
sum(Qloss_GS));
fprintf('\nGauss-Seidel Iterations:  %d\n', iter_GS);
fprintf('Newton-Raphson Iterations: %d\n', iter_NR);

function [Pij, Qij, Pji, Qji, Ploss, Qloss] = calcBranchFlows(V,
brData, Sb)
    nB = size(brData,1);
    Pij = zeros(nB,1);  Qij = zeros(nB,1);
    Pji = zeros(nB,1);  Qji = zeros(nB,1);
    Ploss = zeros(nB,1);  Qloss = zeros(nB,1);
    for k = 1:nB
        p = brData(k,1);  q = brData(k,2);
        r = brData(k,3);  x = brData(k,4);  b = brData(k,5);
        z = r + 1j*x;
        yb = 1/z;
        ysh = 1j*b/2;
        Iij = (V(p) - V(q))*yb + V(p)*ysh;
        Iji = (V(q) - V(p))*yb + V(q)*ysh;
        Sij = V(p)*conj(Iij)*Sb;
        Sji = V(q)*conj(Iji)*Sb;
        Pij(k) = real(Sij);  Qij(k) = imag(Sij);
        Pji(k) = real(Sji);  Qji(k) = imag(Sji);
        Ploss(k) = Pij(k) + Pji(k);
        Qloss(k) = Qij(k) + Qji(k);
    end
end

```

Power Flow Analysis Comparison:

Bus Info		GAUSS-SEIDEL				NEWTON-RAPHSON			
Bus	Type	P (MW)	Q (MVar)	V (pu)	δ (deg)	P (MW)	Q (MVar)	V (pu)	δ (deg)
1	Slack	199.001	27.813	1.060	0	194.548	57.151	1.060	0
2	PV	59.074	-115.908	1.020	-3.532	100.000	11.745	1.020	-3.263
3	PQ	-79.099	56.288	1.059	-11.773	-95.000	-20.000	0.942	-12.072
4	PQ	-42.165	15.699	1.040	-10.210	-45.000	0.000	0.962	-10.353
5	PQ	-8.929	4.926	1.036	-8.804	-10.000	-2.000	0.977	-8.947
6	PQ	-5.281	12.221	1.056	-16.170	-11.000	-7.500	0.984	-18.525

7	PQ	-4.525	6.325	1.079	-14.293	-7.000	-3.390	0.946	-16.006
8	PQ	-17.434	19.242	1.095	-16.091	-25.000	-7.000	0.945	-18.539
9	PQ	-13.160	16.213	1.099	-17.115	-20.000	-6.000	0.936	-19.708
10	PQ	-13.423	15.645	1.086	-17.668	-20.000	-5.000	0.944	-20.185
11	PQ	-17.356	26.435	1.050	-19.140	-30.000	-10.000	0.960	-21.115
12	PV	-27.247	-58.881	1.010	-17.011	6.500	52.585	1.010	-20.816
13	PQ	-13.191	15.842	1.071	-18.091	-20.000	-5.000	0.949	-21.213

Newton-Raphson gives a better load flow solution than Gauss-Seidel (GS). In this 13-bus system, Newton-Raphson converged in just 4 iterations compared to 30 iterations for GS, because Newton-Raphson uses quadratic convergence while GS only converges linearly. GS also required an acceleration factor to avoid slow convergence, whereas Newton-Raphson needed no such tuning and handled stressed buses (Bus 9 at 0.9359 p.u.) more robustly. Both methods produced virtually identical final voltages, confirming accuracy, but Newton-Raphson reached the solution far more efficiently making it the superior and industry-preferred technique for power flow analysis.

Answer to the question “B”

*Matlab Code to observe the line losses after connecting solar PV buses.
(Here we have connected the solar PV buses in the bus no. 2,3,5,10 & 13.)*

```
% EEE 433L Semester Project: 13-Bus Load Flow
% Part (B): Optimal Solar PV Placement
% Student ID: 22221004 | K = 5 | A0 = 50 MW
clear; clc; % Clear workspace and command window
% ----- 1. Data Setup & Y-Bus Formation -----
% Read system data from Excel file
filename = '13bus_21321070.xlsx';
busData = readtable(filename, 'Sheet', 'BusData', 'VariableNamingRule', 'preserve');
lineData = readtable(filename, 'Sheet', 'LineData', 'VariableNamingRule', 'preserve');
genData = readtable(filename, 'Sheet', 'GeneratorData', 'VariableNamingRule', 'preserve');
trafoData = readtable(filename, 'Sheet', 'TransformerData', 'VariableNamingRule', 'preserve');
Sbase = 100;
nbus = height(busData);
% System base power in MVA
% Total number of buses
% Initialize net power injections and voltage parameters
P_net = zeros(nbus, 1); % Net real power injection (pu)
Q_net = zeros(nbus, 1); % Net reactive power injection (pu)
V_target = ones(nbus, 1);
% Voltage magnitude setpoints
type = zeros(nbus, 1); % Bus type: 1=Slack, 2=PV, 3=PQ
```

```

% ----- Bus Data Processing -----
for i = 1:nbus
% Identify bus type
if strcmp(busData.Type{i}, 'Slack Bus')
type(i) = 1;
% Slack bus
elseif strcmp(busData.Type{i}, 'PV') || i == 12
type(i) = 2;
% PV bus
else
end
type(i) = 3;
% PQ bus
% Find generator connected to the current bus
idx = find(genData.Bus == i);
% Extract generator real power (if exists)
Pg = 0;
if ~isempty(idx) && isnumeric(genData.("Pg(MW)")(idx))
Pg = genData.("Pg(MW)")(idx);
end
% Calculate net real and reactive power (pu)
P_net(i) = (Pg - busData.PL(i)) / Sbase;
Q_net(i) = -busData.QL(i) / Sbase;
% Set voltage magnitude for PV and slack buses
if ~isempty(idx)
V_target(i) = genData.("Vg(pu)")(idx);
end
end
% ----- Y-Bus Matrix Formation -----
Y = zeros(nbus, nbus); % Initialize Y-bus matrix
% Contribution from transmission lines
for k = 1:height(lineData)
% Line admittance and shunt admittance
y = 1 / (lineData.R_pu(k) + 1j * lineData.X_pu(k));
b = 1j * lineData.B_pu(k) / 2;
f = lineData.From(k); % From bus
t = lineData.To(k);
% To bus
% Self and mutual admittance updates
Y(f,f) = Y(f,f) + y + b;
Y(t,t) = Y(t,t) + y + b;
Y(f,t) = -y;
Y(t,f) = -y;
end
% Contribution from transformers
for k = 1:height(trafoData)
y = 1 / (trafoData.R_pu(k) + 1j * trafoData.X_pu(k));
f = trafoData.From(k);
t = trafoData.To(k);
Y(f,f) = Y(f,f) + y;
Y(t,t) = Y(t,t) + y;
Y(f,t) = -y;
Y(t,f) = -y;
end
%% ----- 2. Optimal Solar PV Placement -----

```

```

A0_pu = 50 / Sbase;
% Solar PV capacity (70 MW converted to
buses_to_test = [2, 3, 5, 10, 13]; % Candidate buses for PV
placement
results = [];
% Store optimization results
% ----- Base Case (Without Solar PV) -----
V_base
solution = run_gs(Y, type, P_net, Q_net, V_target); % Load flow
base_loss = sum(real(V_base .* conj(Y * V_base))) * Sbase;
% ----- PV Placement at Different Buses -----
for
    b = buses_to_test;
    P_mod = P_net;
    P_mod(b) = P_mod(b) + A0_pu;
    % Run load flow with solar PV
    V_sol = run_gs(Y, type, P_mod, Q_net, V_target);
    % Calculate system losses with PV
    new_loss = sum(real(V_sol .* conj(Y * V_sol))) * Sbase;
    % Percentage loss improvement
    imp = ((base_loss - new_loss) / base_loss) * 100;
    % Store results
    results = [results; b, base_loss, new_loss, imp];
end
%% ----- 3. Results Display -----
fprintf('\n--- PART B: Optimal Solar PV Placement Results ---\n');
PartB_Table = table( ...
    results(:,1), results(:,2), results(:,3), results(:,4), ...
    'VariableNames', {'Solar_PV_Bus', 'Loss_Before_MW', ...
    'Loss_After_MW', 'Improvement_Percent'});
disp(PartB_Table);
%% ----- Gauss-Seidel Load Flow Function
function V = run_gs(Y, type, Pnet, Qnet, V)
    % Iterative Gauss-Seidel solution
    for it = 1:1000

        Vold = V; % Store previous iteration voltages

        for i = 2:length(V) % Skip slack bus

            % Summation term excluding self-admittance
            sumYV = Y(i,:) * V - Y(i,i) * V(i);

            if type(i) == 3 % PQ Bus

                % Voltage update for PQ bus
                V(i) = (1 / Y(i,i)) * ...
                    ((Pnet(i) - 1j * Qnet(i)) / conj(V(i)) -
sumYV);

            else % PV Bus

                % Calculate reactive power for PV bus
                Qi = -imag(conj(V(i)) * (sumYV + Y(i,i) * V(i)));

                % Voltage update

```

```

        V(i) = (1 / Y(i,i)) * ...
            ((Pnet(i) - 1j * Qi) / conj(V(i)) - sumYV);

        % Enforce voltage magnitude constraint
        V(i) = abs(V(i)) * (V(i) / abs(V(i)));
    end
end

% Convergence check
if max(abs(V - Vold)) < 1e-6
    break;
end
end
end
end

```

Answer to the question “C”

Constructing the given 13 bus system in PV simulator, we have network diagrams like these;

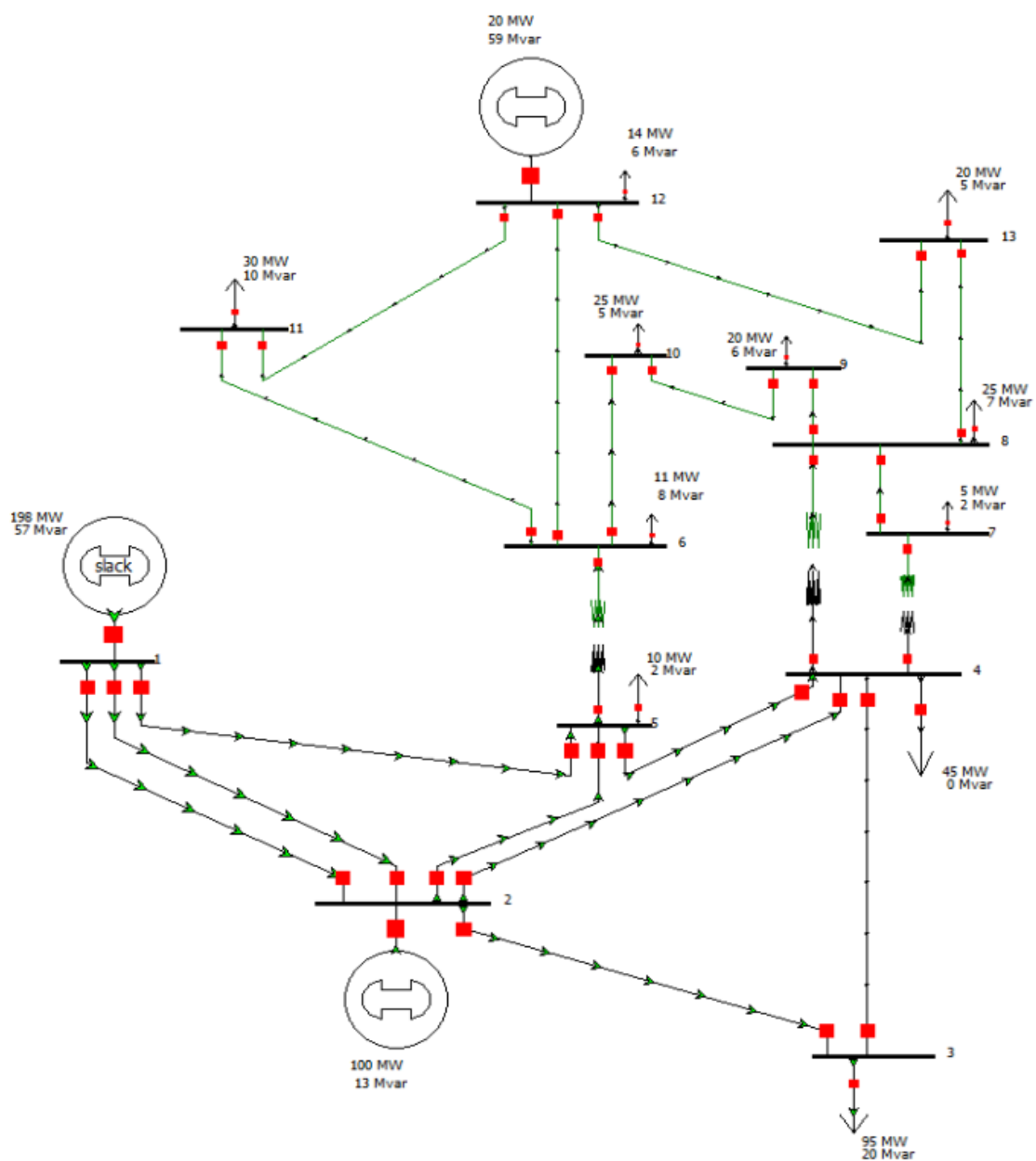


Fig01: 13 bus network diagram **without solar bus** simulated by gauss seidel method

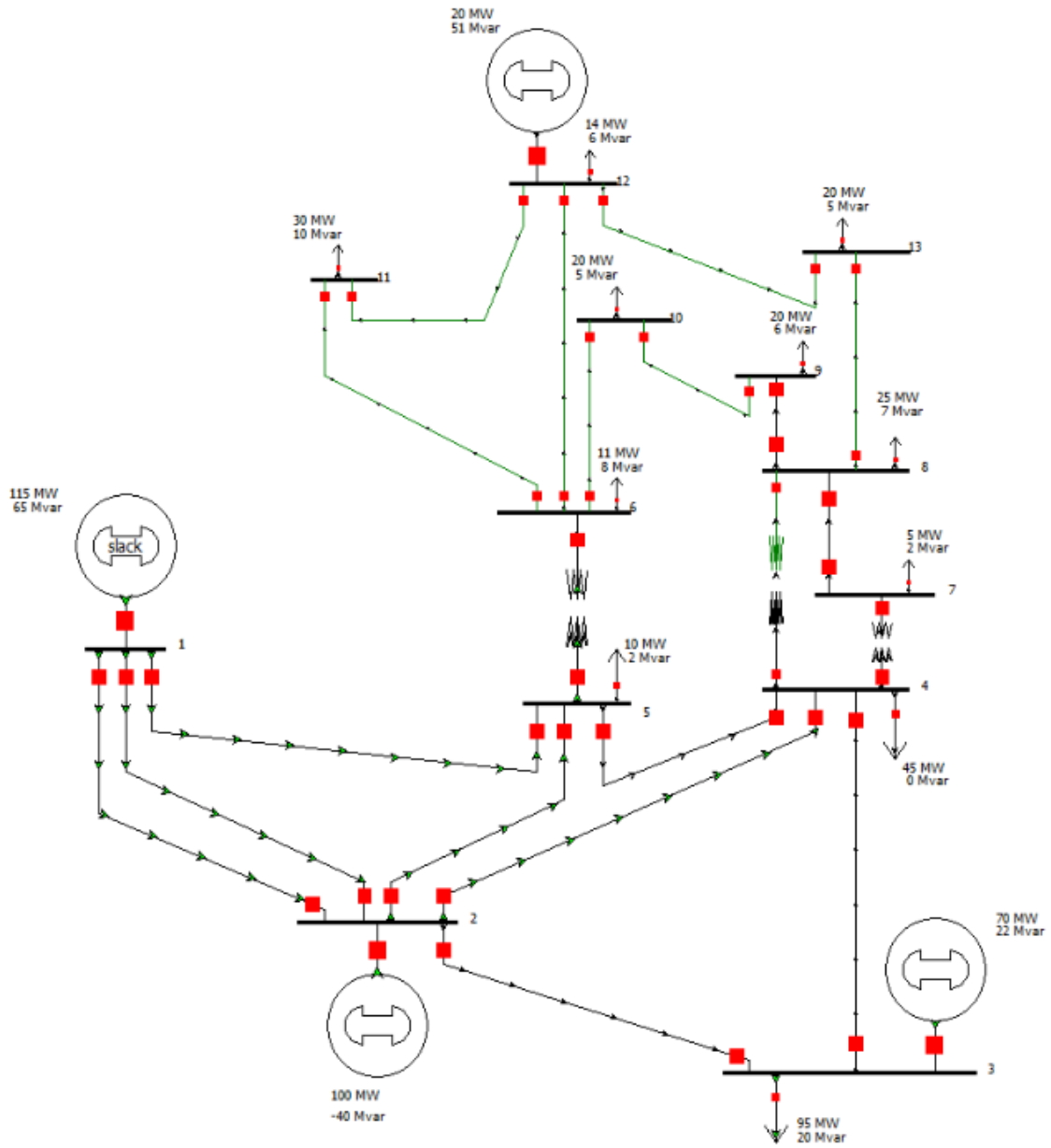


Fig02: 13 bus network diagram **with solar bus (in bus no.3)** simulated by gauss seidel method

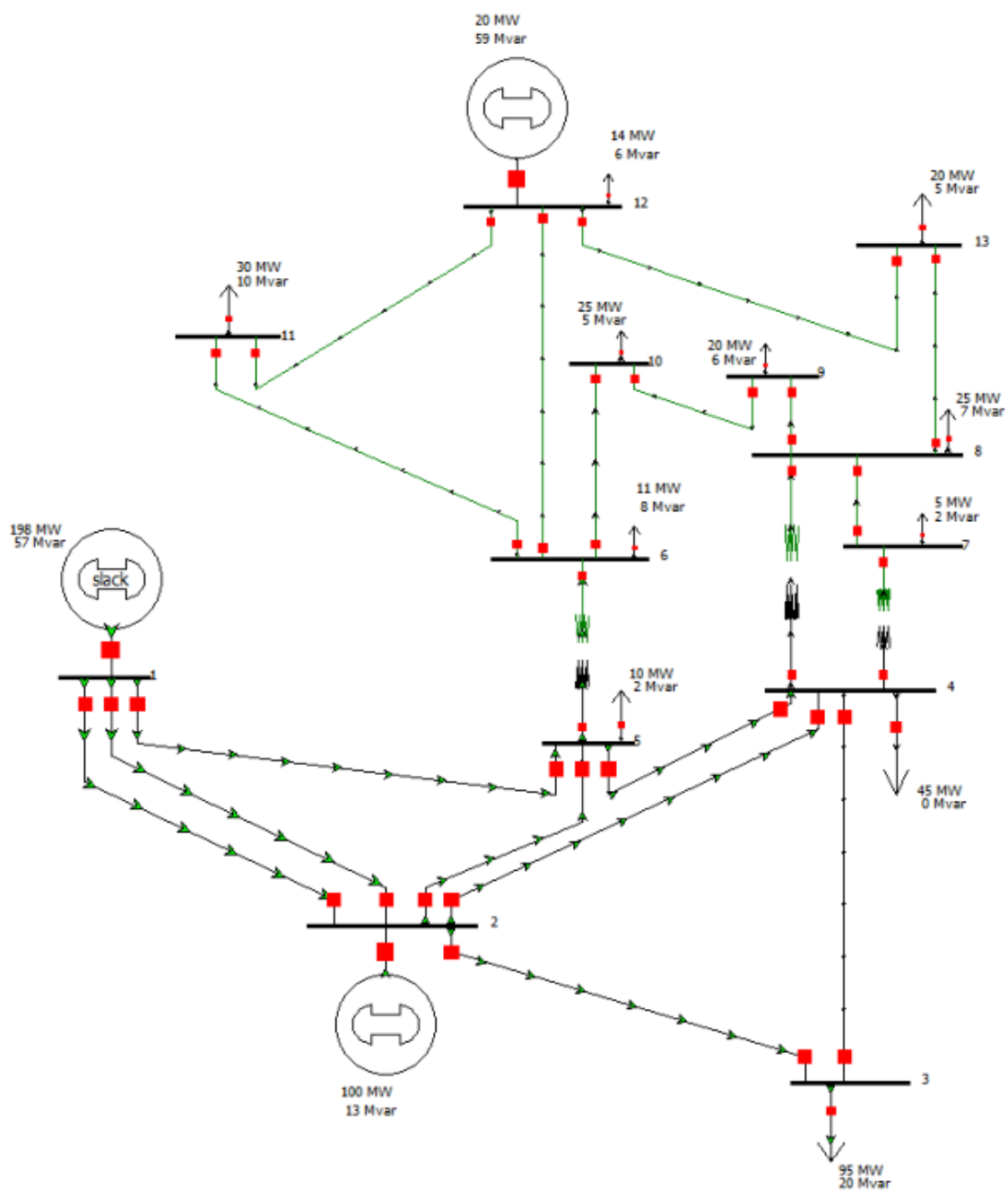


Fig03: 13 bus network diagram **without solar bus** simulated by Newton Raphson method

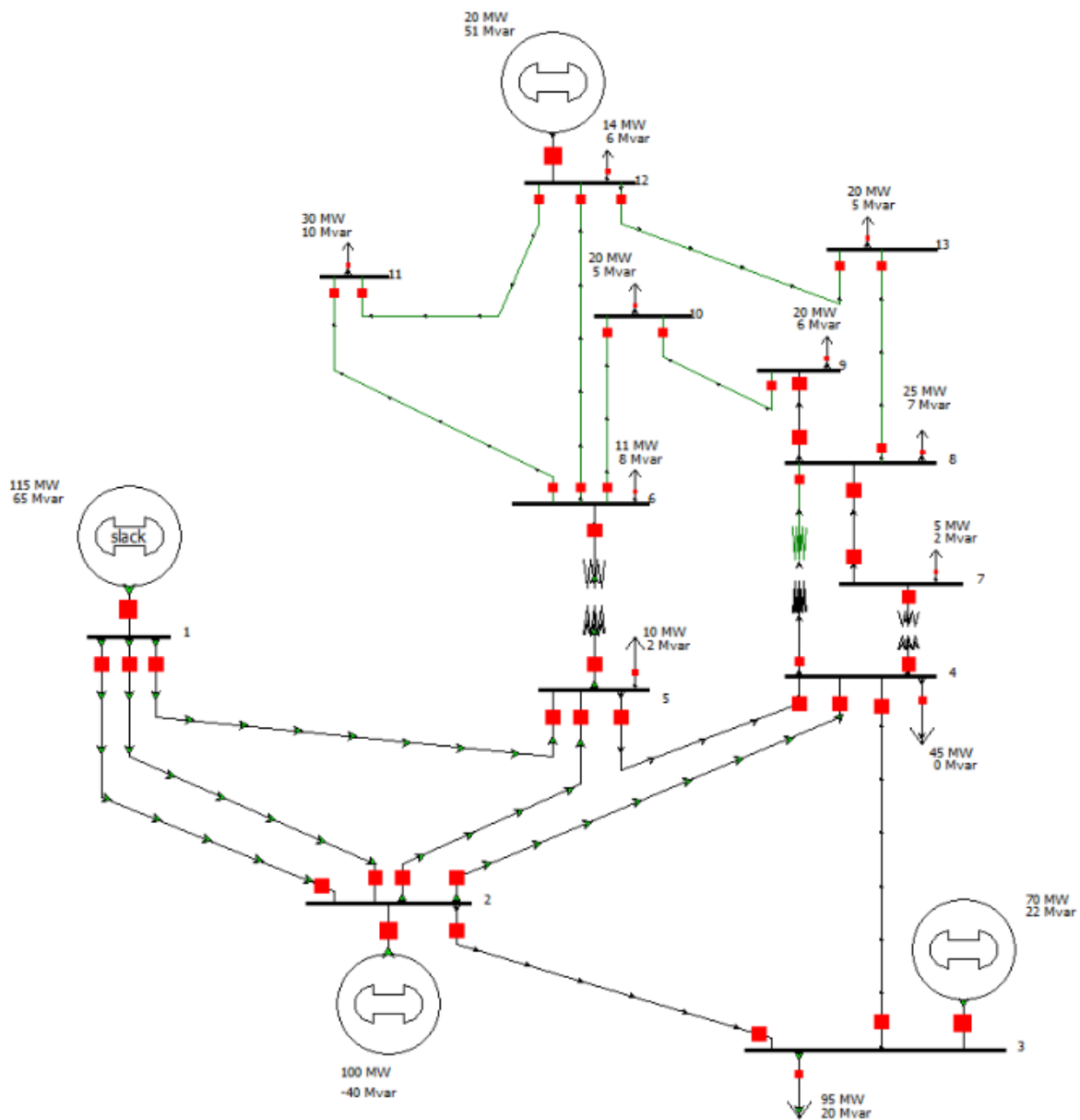


Fig04: 13 bus network diagram **with solar bus (in bus no.3)** simulated by Newton Raphson method

As we have found the same results for both Gauss Seidel and Newton Raphson method, just the results for the cases of solar PV buses and without solar PV buses are given below;

	Number	Name	Area Name	Nom kV	PU Volt	Volt (kV)	Angle (Deg)	Gen MW	Gen Mvar	Load MW	Load Mvar
1	1 1	1	1	138.00	1.06000	146.280	0.00	198.09	57.21		
2	2 2	1	1	138.00	1.02000	140.760	-3.34	100.00	13.36		
3	3 3	1	1	138.00	0.94123	129.889	-12.20			95.00	20.00
4	4 4	1	1	138.00	0.96168	132.712	-10.52			45.00	0.00
5	5 5	1	1	138.00	0.97563	134.637	-9.11			10.00	2.00
6	6 6	1	1	66.00	0.98287	64.870	-19.05			11.00	7.50
7	7 7	1	1	66.00	0.94628	62.454	-16.22			5.00	2.42
8	8 8	1	1	66.00	0.94454	62.340	-18.92			25.00	7.00
9	9 9	1	1	66.00	0.93482	61.698	-20.22			20.00	6.00
10	10 10	1	1	66.00	0.94073	62.088	-21.02			25.00	5.00
11	11 11	1	1	66.00	0.95947	63.325	-21.64			30.00	10.00
12	12 12	1	1	66.00	1.01000	66.660	-21.34	20.00	59.41	13.50	6.00
13	13 13	1	1	66.00	0.94919	62.647	-21.66			20.00	5.00

Fig05 : Results **without the solar PV** buses

	Number	Name	Area Name	Nom kV	PU Volt	Volt (kV)	Angle (Deg)	Gen MW	Gen Mvar	Load MW	Load Mvar
1	1 1	1	1	138.00	1.06000	146.280	0.00	114.80	65.08		
2	2 2	1	1	138.00	1.02274	141.138	-1.29	100.00	-40.00		
3	3 3	1	1	138.00	1.00000	138.000	-5.83	70.00	21.82	95.00	20.00
4	4 4	1	1	138.00	0.98141	135.434	-7.23			45.00	0.00
5	5 5	1	1	138.00	0.99047	136.685	-6.40			10.00	2.00
6	6 6	1	1	66.00	0.98879	65.260	-15.41			11.00	7.50
7	7 7	1	1	66.00	0.96275	63.542	-12.65			5.00	2.42
8	8 8	1	1	66.00	0.95899	63.293	-15.23			25.00	7.00
9	9 9	1	1	66.00	0.94890	62.627	-16.41			20.00	6.00
10	10 10	1	1	66.00	0.95354	62.934	-16.97			20.00	5.00
11	11 11	1	1	66.00	0.96222	63.507	-17.90			30.00	10.00
12	12 12	1	1	138.00	1.01000	139.380	-17.45	20.00	50.92	13.50	6.00
13	13 13	1	1	66.00	0.95759	63.201	-17.84			20.00	5.00

Fig06 : Results **with the solar PV** buses

By changing the power flow dynamics from a net-load bus to a local generation support (PV/PQ or PV), integrating the solar PV at Bus-3 reduces the upstream power transfer. Because part of the demand is now met locally at Bus-3, slack bus generation ($\approx 198-115$ MW) decreases. The reduction in power transfer also means there is a lesser cumulative angle drop throughout the network, leading to voltage angles at downstream buses becoming less negative (enhanced angular stability profile). In the absence of the PV, Bus-3 consumes power, which raises line loading and losses, compelling the slack generator to produce more MW and creating larger phase angle differences. In summary, the PV helps generation distribution, reduces pressure on the slack bus, reduces transmission strain, and improves the voltage angle profile, aligning with the anticipated behavior of distributed generation in radial or weakly meshed systems.

Conclusion:

In this project, load flow analysis of a 13-bus power system was successfully carried out using both MATLAB and PowerWorld Simulator. This study helped understand practically, power flow analysis, Y-bus matrix formation, bus classification, voltage profile analysis and integration of renewable energy. The comparison showed that the Newton–Raphson method was more convergent, numerically stable and gave more accurate results for the given system. With 50 MW solar PV source connected at Bus-3, it was observed that the voltage profile was improved and load percentage of the slack bus was reduced, which resulted in the better power support in the network level. The results of the simulation also showed the benefits of the overall system operation and distribution of power flow. The project faced a few challenges and potential issues. A major challenge was the correct formation of the Y-bus admittance matrix from the branch and transformer data as even a small error in either impedance or admittance values resulted in incorrect load flow results. Another challenge was dealing with convergence problems with the Gauss–Seidel method during heavy load conditions because the method was required to run for a greater number of iterations and it was found to be sensitive to the initial voltage assumptions. It was also difficult to debug the MATLAB code, to keep the per-unit values consistent and to get the correct classification of slack, PV, and PQ buses. To prevent unrealistic operating conditions or simulation mismatch, careful adjustments of generator settings, voltage limits and transformer configurations were necessary in PowerWorld simulation. In order to solve these issues, we have implemented some solutions. Hardcoded errors were eliminated and flexibility in changing system parameters was improved by using the Excel based input data.

Accurate verification of branch data, transformer parameters and bus classification resulted in correct Y-bus formation. The acceleration factors and appropriate tolerance values were used in the Gauss–Seidel method to minimize convergence problems and Newton–Raphson method was chosen for faster and more reliable convergence. The correctness of the analysis was also checked by comparing the MATLAB results with the results obtained from PowerWorld simulation. In conclusion, the project offered hands-on experience in power system analysis and illustrated the benefits of integrating renewable energy sources in enhancing the efficiency, reducing losses, and enhancing voltage stability of modern power systems.